

CODING TECHNICAL ASSESSMENT

Position: Full Stack Java Developer

Level: Mid-level

Technologies: Java, HTML, CSS, JavaScript

INSTRUCTIONS:

- This is a coding-focused technical assessment
- Complete all coding exercises and projects
- Submit your code files with clear documentation
- Time limit: 2-3 hours
- Reply to this email with your completed solutions

****Full Stack Java Developer Coding Assessment****

****Time Limit:** 3 hours**

****Instructions:****

- Please submit your code in a ZIP file named as
`YourName\_FullStackJavaDeveloper.zip`.
- Use a Java IDE (e.g., Eclipse, IntelliJ) for coding exercises.
- Use a text editor or IDE (e.g., Visual Studio Code) for front-end coding exercises.
- Use a database management tool (e.g., MySQL Workbench) for database query challenges.
- Include all necessary dependencies, libraries, and configuration files.
- Make sure to test your code before submitting it.

****File Naming Conventions:****

- Use the problem number as the file name (e.g., `problem1.java`, `problem2.html`, etc.).
- Use the project name as the folder name (e.g., `project1`, `project2`, etc.).

****How to Run and Test the Code:****

- For Java coding exercises, use the ``javac`` command to compile and the ``java`` command to run the code.
- For front-end coding exercises, open the HTML file in a web browser to test the code.
- For database query challenges, use a database management tool to execute the SQL queries.

****Submission Guidelines:****

- Submit your code as a ZIP file via email or upload it to a cloud storage service (e.g., Google Drive, Dropbox).
- Include your name, contact information, and a brief description of your solutions in the email or submission notes.

****CODING EXERCISES (7 questions)****

Problem 1: Algorithm and Data Structure (30 minutes)

Implement a Java method to find the first duplicate in an array of integers. The method should return the first duplicate element, or -1 if no duplicates are found.

****Input/Output Examples:****

- Input: ``[1, 2, 3, 4, 5, 2]``
- Output: ``2``
- Input: ``[1, 2, 3, 4, 5]``
- Output: ``-1``

****Evaluation Criteria:****

- Correctness (40%)
- Time complexity (30%)
- Code readability and maintainability (30%)

****Sample Test Cases:****

- `[1, 2, 3, 4, 5, 2]`
- `[1, 2, 3, 4, 5]`
- `[5, 4, 3, 2, 1]`
- `[]`

Problem 2: Code Implementation Challenge (30 minutes)

Implement a Java class to represent a bank account. The class should have methods to deposit, withdraw, and check the account balance. The class should also handle invalid operations (e.g., withdrawing more than the available balance).

****Input/Output Examples:****

- Input: `account.deposit(100); account.withdraw(50);`
- Output: `50`
- Input: `account.withdraw(100);`
- Output: `Invalid operation: Insufficient funds`

****Evaluation Criteria:****

- Correctness (40%)
- Code organization and structure (30%)
- Error handling (30%)

****Sample Test Cases:****

- `account.deposit(100); account.withdraw(50);`
- `account.withdraw(100);`
- `account.deposit(-50);`
- `account.withdraw(-50);`

Problem 3: Bug Fixing Exercise (30 minutes)

Fix the bug in the following Java code:

```
```java
public class Calculator {
 public int add(int a, int b) {
 return a - b;
 }
}
```
```

The code should correctly add two integers.

****Input/Output Examples:****

- Input: `calculator.add(2, 3)`
- Output: `5`

****Evaluation Criteria:****

- Correctness (40%)
- Code readability and maintainability (30%)
- Debugging skills (30%)

****Sample Test Cases:****

- `calculator.add(2, 3)`
- `calculator.add(-2, 3)`
- `calculator.add(0, 0)`

Problem 4: API Development Task (45 minutes)

Implement a RESTful API using Java to manage a collection of books. The API should have endpoints to add, remove, and retrieve books.

****Input/Output Examples:****

- Input: `POST /books {"title": "Book Title", "author": "Author Name"}`
- Output: `201 Created`
- Input: `GET /books`
- Output: `[{"title": "Book Title", "author": "Author Name"}]`

****Evaluation Criteria:****

- Correctness (40%)
- API design and documentation (30%)
- Code organization and structure (30%)

****Sample Test Cases:****

- `POST /books {"title": "Book Title", "author": "Author Name"}`
- `GET /books`
- `DELETE /books/1`

Problem 5: Front-end Coding Exercise (30 minutes)

Implement an HTML form to collect user input for a simple login page. The form should have fields for username and password, and a submit button.

****Input/Output Examples:****

- Input: `username: "john", password: "hello"`
- Output: `Login successful`

****Evaluation Criteria:****

- Correctness (40%)
- HTML structure and organization (30%)
- CSS styling (30%)

****Sample Test Cases:****

- `username: "john", password: "hello"`
- `username: "", password: "hello"`
- `username: "john", password: ""`

Problem 6: Database Query Challenge (45 minutes)

Write a SQL query to retrieve the top 5 highest-paid employees from the `employees` table.

****Table Schema:****

- `employees` table:
 - `id` (primary key)
 - `name`
 - `salary`

****Input/Output Examples:****

- Output: `["John Doe", "Jane Doe", "Bob Smith", "Alice Johnson", "Mike Brown"]`

****Evaluation Criteria:****

- Correctness (40%)
- Query efficiency (30%)
- SQL syntax and organization (30%)

****Sample Test Cases:****

- `SELECT \* FROM employees WHERE salary > 50000`
- `SELECT \* FROM employees WHERE name LIKE "%Doe%"`
- `SELECT \* FROM employees WHERE id = 1`

Problem 7: Code Optimization Task (30 minutes)

Optimize the following Java code to improve its performance:

```
```java
public class StringRepeater {
 public String repeatString(String str, int count) {
 String repeatedStr = "";
 for (int i = 0; i < count; i++) {
 repeatedStr += str;
 }
 return repeatedStr;
 }
}
```
```

The code should repeat a given string a specified number of times.

****Input/Output Examples:****

- Input: `repeater.repeatString("hello", 3)`
- Output: `"hellohellohello"`

****Evaluation Criteria:****

- Correctness (40%)
- Performance improvement (30%)
- Code readability and maintainability (30%)

****Sample Test Cases:****

- `repeater.repeatString("hello", 3)`
- `repeater.repeatString("", 5)`
- `repeater.repeatString("hello", 0)`

****PRACTICAL PROJECTS (2 mini-projects)****

Project 1: Simple Blog Application (60 minutes)

Implement a simple blog application using Java, HTML, and CSS. The application should have a home page, a blog post page, and a contact page. The application should also have a navigation menu and a footer.

****Requirements:****

- Use Java to generate the HTML content
- Use CSS to style the application
- Use HTML to structure the application

****Evaluation Criteria:****

- Correctness (40%)
- Code organization and structure (30%)
- User experience and design (30%)

Project 2: To-Do List Application (60 minutes)

Implement a to-do list application using Java, HTML, and CSS. The application should have a form to add new tasks, a list to display existing tasks, and a button to delete completed tasks.

****Requirements:****

- Use Java to handle form submissions and update the task list
- Use CSS to style the application
- Use HTML to structure the application

****Evaluation Criteria:****

- Correctness (40%)
- Code organization and structure (30%)

- User experience and design (30%)

****CODE REVIEW TASKS (3 questions)****

Task 1: Code Review (30 minutes)

Review the following Java code and suggest improvements:

```
```java
public class Calculator {
 public int add(int a, int b) {
 return a + b;
 }
}
```
```

The code should add two integers.

****Evaluation Criteria:****

- Code readability and maintainability (40%)
- Performance improvement (30%)
- Error handling (30%)

Task 2: Code Refactoring (30 minutes)

Refactor the following Java code to improve its performance and readability:

```
```java
public class StringRepeater {
 public String repeatString(String str, int count) {
 String repeatedStr = "";
 for (int i = 0; i < count; i++) {
 repeatedStr += str;
 }
 return repeatedStr;
 }
}
```

```
}
...

```

The code should repeat a given string a specified number of times.

**\*\*Evaluation Criteria:\*\***

- Correctness (40%)
- Performance improvement (30%)
- Code readability and maintainability (30%)

**### Task 3: Code Review and Improvement (45 minutes)**

Review the following Java code and suggest improvements:

```
```java  
public class BankAccount {  
    private int balance;  
  
    public BankAccount(int balance)  

```

Generated on: 15/09/2025 14:28:46